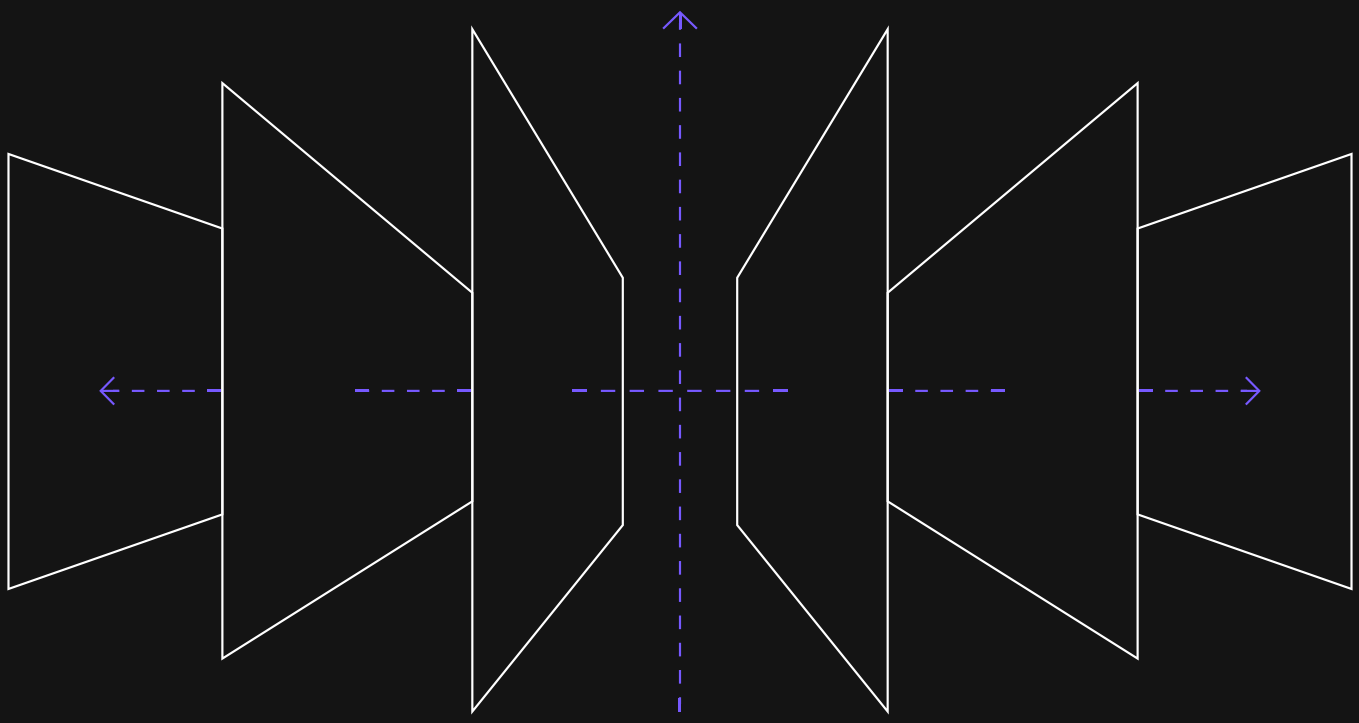
Машинное обучение

Лонгрид

Глубокое обучение

08

# Современные трансформеры: RoPE, GQA, MLA и MoE



# Современные трансформеры

Рассмотрим decoder-only transformer при авторегрессионном инференсе. Требуется задать позиционную зависимость attention, уменьшить объем памяти, занимаемый KV-cache и увеличить емкость FFN при ограниченном объеме вычислений на токен.

Рассматриваются четыре механизма. RoPE кодирует положение токена с помощью поворота запросов и ключей. MQA и GQA уменьшают число наборов ключей и значений. MLA хранит низкоразмерное представление ключей и значений. MoE заменяет один плотный FFN набором экспертов, из которых для токена активируется только часть. Для каждого механизма фиксируются базовая операция, изменение формулы и проверяемое свойство.

## Содержание

**1 Постановка задачи и обозначения** **2**

**2 Rotary Position Embedding** **3**

**3 Multi-query и grouped-query attention** **4**

**4 Multi-head Latent Attention** **5**

**5 Mixture of Experts** **7**

**6 Положение механизмов в блоке трансформера** **9**

**7 Оценка качества и эффективности** **10**

**8 Основные выводы** **10**

**A Вывод относительного сдвига в RoPE** **11**

**B Пример расчета KV-cache** **11**

**Литература** **12**

# 1. Постановка задачи и обозначения

Рассмотрим один слой причинного трансформера. На входе находятся состояния токенов

$$H = (h_1, \dots, h_T)^\top \in \mathbb{R}^{T \times d},$$

где $T$ - длина последовательности, $d$ - размер состояния. Выход слоя имеет ту же форму. В модели $L$ слоев. В multi-head attention используется $n_h$ голов размерности $d_h$; обычно $d = n_h d_h$. Размер батча обозначим через $B$, число байтов на один элемент кеша - через $s$.

Для головы $i$ и позиции $t$ стандартные проекции имеют вид

$$q_{t,i} = W_i^Q h_t, \qquad k_{t,i} = W_i^K h_t, \qquad v_{t,i} = W_i^V h_t,$$

где $q_{t,i}, k_{t,i}, v_{t,i} \in \mathbb{R}^{d_h}$. Причинное внимание вычисляет

$$a_{t,j,i} = \frac{\exp(q_{t,i}^\top k_{j,i}/\sqrt{d_h})}{\sum_{r=1}^{t} \exp(q_{t,i}^\top k_{r,i}/\sqrt{d_h})}, \qquad o_{t,i} = \sum_{j=1}^{t} a_{t,j,i} v_{j,i}.$$

Индекс $j$ пробегает доступные позиции префикса. Матрицы $W_i^Q, W_i^K, W_i^V$ обучаются. При генерации состояние нового токена меняется на каждом шаге, а ключи и значения уже обработанного префикса можно сохранить.

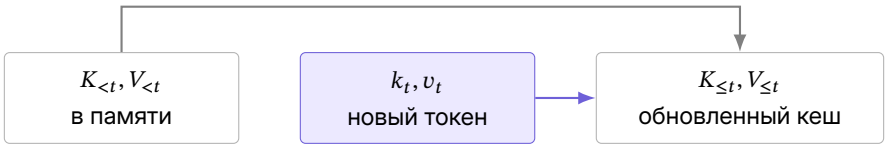



**Рис. 1:** Ключи и значения нового токена добавляются к сохраненным представлениям префикса. Запрос текущего токена в кеш не записывается [3].

Для обычного multi-head attention кеш одного слоя содержит $Tn_h d_h$ чисел для ключей и столько же для значений. Для всего батча и всех слоев объем равен

$$M_{\text{MHA}} = 2BTLn_h d_h s = 2BTLds \quad \text{байт.}$$

Множитель 2 соответствует ключам и значениям. Запросы хранить для следующих шагов не требуется: запрос строится только для текущей позиции. Кеш растет линейно по $B$, $T$ и $L$ и часто ограничивает размер батча или длину контекста при декодировании.

Вторая крупная часть блока - позиционный FFN. В плотном варианте один и тот же набор параметров применяется к каждому токену. Если промежуточная размерность равна $d_{\text{ff}}$, число параметров и число операций FFN растут примерно как $dd_{\text{ff}}$. В MoE число доступных FFN увеличивается, но для одного токена вычисляются только выбранные эксперты.

Методы решают три разные задачи:

- RoPE задает позиционную зависимость оценок внимания, но не сокращает кеш;
- GQA и MLA уменьшают объем состояния, сохраняемого при генерации;
- MoE изменяет FFN: увеличивает число доступных параметров, вычисляя для каждого токена только выбранных экспертов.

Качество архитектуры нельзя оценивать только числом параметров. Нужны как минимум потеря или метрика на отложенных данных, объем памяти, скорость обработки входа, скорость генерации и пропускная способность при фиксированном оборудовании.

# 2. Rotary Position Embedding

## 2.1. Требование к позиционному кодированию

Проекции запросов и ключей сами по себе не содержат номера позиций. Причинная маска задает допустимое направление связи, но не добавляет расстояние $n - m$ в оценку пары. Требуется ввести позиционное преобразование, при котором оценка связи между позициями $m$ и $n$ зависит от их относительного расстояния. Для запросов и ключей это условие можно записать как

$$\langle f_Q(h_m, m), f_K(h_n, n)\rangle = g(h_m, h_n, n - m).$$

RoPE строит функции $f_Q$ и $f_K$ при помощи поворотов в двумерных подпространствах [2].

Возьмем пару координат $z = (z_1, z_2)^\top$ и матрицу поворота

$$R(m\theta) = \begin{pmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{pmatrix}.$$

Для четной размерности $d_h$ разобьем вектор на $d_h/2$ пар. У каждой пары своя частота

$$\theta_r = b^{-2r/d_h}, \qquad r = 0, \dots, d_h/2 - 1,$$

где в исходной работе $b = 10^4$. Блочно-диагональную матрицу из поворотов обозначим через $R_m$. Тогда

$$\tilde{q}_m = R_m W^Q h_m, \qquad \tilde{k}_n = R_n W^K h_n.$$

Значения $v_n$ не вращаются: позиция влияет на веса внимания, а не непосредственно на складываемые векторы.

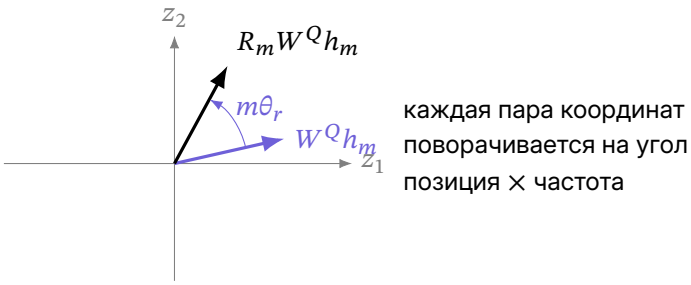



**Рис. 2:** Одна двумерная компонента RoPE. В полной голове одновременно используются $d_h/2$ частот.

Главное тождество следует из ортогональности поворота:

$$\tilde{q}_m^\top \tilde{k}_n = (W^Q h_m)^\top R_m^\top R_n (W^K h_n) = (W^Q h_m)^\top R_{n-m} (W^K h_n).$$

Абсолютные номера $m$ и $n$ входят в отдельные повороты, но их скалярное произведение зависит от разности позиций. Именно это свойство требуется оценке внимания. Норма запроса и ключа при повороте не меняется.

## 2.2. Вычисление и ограничения

Полную матрицу $R_m$ в реализации не строят. Для каждой пары координат вычисляют

$$z'_{2r} = z_{2r}\cos(m\theta_r) - z_{2r+1}\sin(m\theta_r),$$
$$z'_{2r+1} = z_{2r}\sin(m\theta_r) + z_{2r+1}\cos(m\theta_r).$$

Значения синусов и косинусов для нужных позиций можно подготовить заранее. Параметры частот в базовой версии не обучаются, но проекции запросов и ключей обучаются обычным образом.

RoPE не снимает ограничение на длину контекста автоматически. Если на обучении модель видела позиции до $T_{\text{train}}$, работа на существенно больших номерах зависит от частот, данных и способа продолжения позиционной шкалы. Методы изменения базы или масштабирования позиции решают отдельную задачу и должны проверяться на длинных последовательностях. Само равенство с $R_{n-m}$ не гарантирует сохранения качества вне обучающего диапазона.

# 3. Multi-query и grouped-query attention

## 3.1. Число голов ключей и значений

В MHA каждой из $n_h$ голов запросов соответствует собственная голова ключей и значений. Это дает $n_h$ разных наборов $K$ и $V$, но требует хранить их все. Multi-query attention оставляет $n_h$ голов запросов и использует одну общую голову ключей и одну общую голову значений [3].

Grouped-query attention выбирает промежуточный вариант [4]. Пусть $n_g$ - число групп ключей и значений, причем $n_g$ делит $n_h$. Голове запроса $i$ соответствует группа

$$g(i) = 1 + \left\lfloor \frac{(i-1)n_g}{n_h} \right\rfloor.$$

Для этой головы

$$q_{t,i} = W_i^Q h_t, \qquad k_{t,g(i)} = W_{g(i)}^K h_t, \qquad v_{t,g(i)} = W_{g(i)}^V h_t.$$

Запросы остаются различными, а ключи и значения разделяются между головами одной группы.

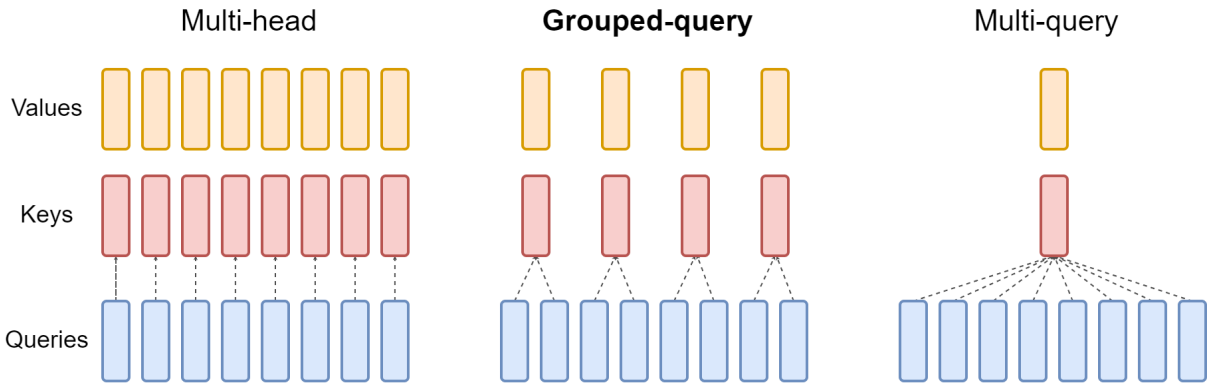



**Рис. 3:** MHA, GQA и MQA для восьми голов запросов. В средней схеме четыре группы ключей и значений. Ainslie et al., рис. 2 [4].

Крайние случаи дают знакомые методы:

$$n_g = n_h \implies \text{MHA}, \qquad n_g = 1 \implies \text{MQA}.$$

Объем кеша для GQA равен

$$M_{\text{GQA}} = 2BTLn_g d_h s.$$

Отношение к MHA составляет $n_g/n_h$. Например, при $n_h = 32$ и $n_g = 8$ кеш ключей и значений уменьшается в четыре раза. Число голов запросов и размер выхода attention при этом не меняются.

### 3.2. Компромисс качества и скорости

Уменьшение $n_g$ заставляет больше голов запросов пользоваться одним представлением ключей и значений. Это сокращает кеш и объем данных, который требуется читать на каждом шаге генерации, но ограничивает независимость голов. MQA дает максимальное сокращение, GQA позволяет выбрать промежуточную точку.

В работе GQA также предложено преобразовывать обученную MHA-модель: матрицы ключей и значений внутри группы усредняются, после чего модель дополнительно предобучается. Авторы сообщают, что такой GQA сохраняет качество, близкое к MHA, при скорости, близкой к MQA, в их экспериментальной постановке [4]. Это экспериментальный результат конкретной работы, а не тождество для любой модели.

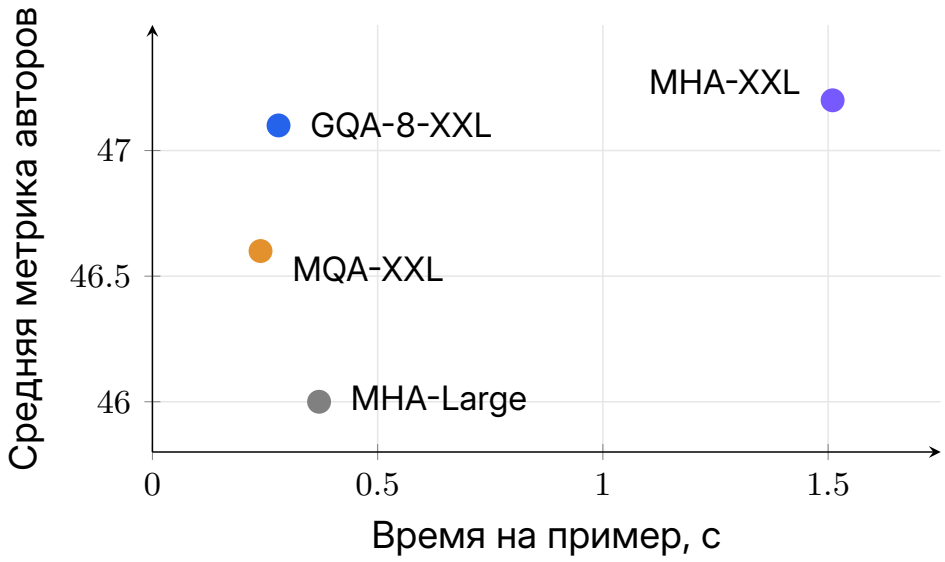



**Рис. 4:** Средняя метрика качества и время на пример для моделей T5 в эксперименте GQA. График построен по числам таблицы 1; время указано в секундах. Измерения на TPUv4 относятся к режиму авторов [4].

При сравнении вариантов нужно фиксировать число голов запросов, размер модели, длину контекста, числовой формат и реализацию ядра attention. Отдельно измеряются обработка всего входа и пошаговое декодирование. GQA прежде всего сокращает трафик KV-cache при декодировании; выигрыш во времени обработки всего входного текста может быть меньше.

## 4. Multi-head Latent Attention

### 4.1. Совместное сжатие ключей и значений

GQA уменьшает число голов $K$ и $V$, разделяя их между запросами. MLA использует другую параметризацию: для каждого токена хранится общий низкоразмерный вектор, из которого восстанавливаются ключи и значения всех голов [5].

Пусть $d_c$ - размер сжатого представления, причем $d_c \ll n_h d_h$. Для состояния $h_t$ вычисляется

$$c_t^{KV} = W^{DKV} h_t, \qquad c_t^{KV} \in \mathbb{R}^{d_c}.$$

Матрица $W^{DKV} \in \mathbb{R}^{d_c \times d}$ выполняет понижение размерности. Полные ключи и значения получают обратными проекциями:

$$k_t^C = W^{UK} c_t^{KV}, \qquad v_t^C = W^{UV} c_t^{KV},$$

где $k_t^C, v_t^C \in \mathbb{R}^{n_h d_h}$. Верхний индекс $C$ отмечает содержательную часть, построенную из сжатого вектора. На инференсе сохраняется $c_t^{KV}$, а не оба полных вектора.

Для запросов в DeepSeek-V2 также используется низкоранговая проекция размерности $d'_c$:

$$c_t^Q = W^{DQ} h_t, \qquad c_t^Q \in \mathbb{R}^{d'_c},$$

$$q_t^C = W^{UQ} c_t^Q.$$

Она уменьшает объем промежуточных активаций на обучении. Для KV-cache эта проекция несущественна, потому что запрос текущего токена не сохраняется.

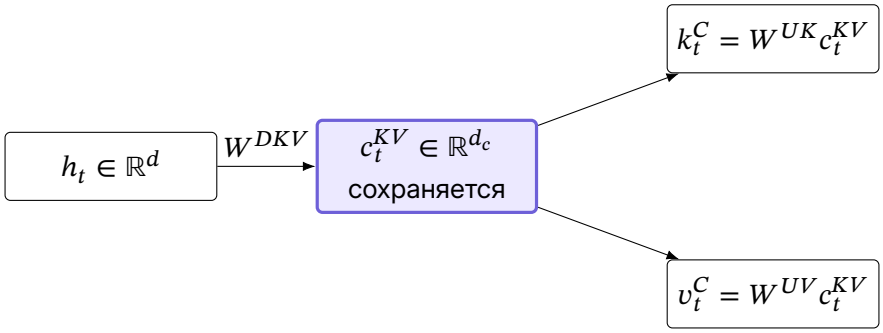



**Рис. 5:** Совместное низкоранговое представление ключей и значений в MLA [5].

Без позиционной части кеш на один токен и слой содержал бы $d_c$ чисел вместо $2n_h d_h$. Однако стандартное применение RoPE ко всему восстановленному ключу мешает вычислительной перестановке матриц: между $W^{UK}$ и запросом появляется зависящий от позиции поворот. Поэтому MLA отделяет позиционные координаты.

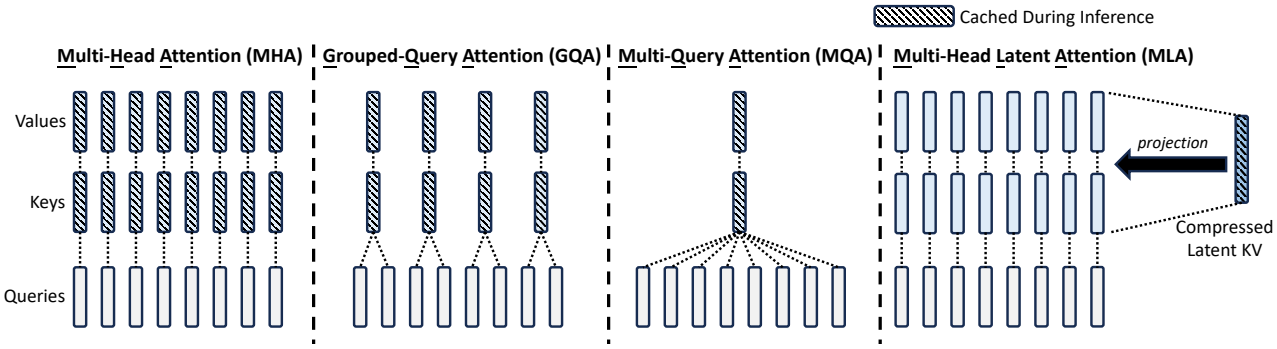



**Рис. 6:** Сравнение представлений ключей и значений. Штриховкой отмечены сохраняемые при инференсе величины. В MLA позиционный ключ добавляется отдельно; он разобран в следующем подразделе. DeepSeek-V2, рис. 3 [5].

## 4.2. Отделенная позиционная часть

Для каждой головы запроса строятся позиционные координаты $q_{t,i}^R \in \mathbb{R}^{d_h^R}$, а для ключа используется общий вектор $k_t^R \in \mathbb{R}^{d_h^R}$:

$$[q_{t,1}^R; \ldots; q_{t,n_h}^R] = \mathrm{RoPE}_t(W^{QR} c_t^Q), \qquad k_t^R = \mathrm{RoPE}_t(W^{KR} h_t).$$

После разбиения содержательных векторов по головам полные запрос и ключ задаются конкатенацией

$$q_{t,i} = [q^C_{t,i}; q^R_{t,i}], \qquad k_{t,i} = [k^C_{t,i}; k^R_t].$$

Веса и выход головы имеют вид

$$a_{t,j,i} = \mathrm{softmax}_j\left(\frac{q^\top_{t,i} k_{j,i}}{\sqrt{d_h + d^R_h}}\right), \qquad o_{t,i} = \sum_{j=1}^{t} a_{t,j,i} v^C_{j,i}.$$

Позиционная часть участвует в скалярном произведении, но не добавляется к значению. В кеше нужно хранить $c^{KV}_t$ и $k^R_t$. Поэтому

$$M_{\mathrm{MLA}} = BTL(d_c + d^R_h)s.$$

Это выражение нужно сравнивать с $2BTLn_h d_h s$ для MHA и $2BTLn_g d_h s$ для GQA при одинаковых $B$, $T$, $L$ и формате чисел.

В статье DeepSeek-V2 отмечено, что блок матрицы восстановления ключа для головы $i$ можно поглотить в проекцию запроса:

$$(q^C_i)^\top W^{UK}_i c^{KV} = \left((W^{UK}_i)^\top q^C_i\right)^\top c^{KV}.$$

Матрица восстановления значений аналогично объединяется с выходной проекцией. После такой перегруппировки полные $k^C_t$ и $v^C_t$ не обязаны материализовываться на каждом шаге. Сжатый размер KV-cache сохраняется и без этой перегруппировки, но восстановление полных ключей и значений требует дополнительного времени и памяти для промежуточных активаций. Свойство поглощения зависит от линейности проекций; именно поэтому RoPE вынесен в отдельную часть.

MLA не является простой заменой числа голов в готовой MHA-модели. Меняется параметризация attention, появляются размеры $d_c$, $d'_c$ и $d^R_h$, а эффективная реализация требует соответствующих ядер. Сравнение с GQA должно учитывать не только формальный размер кеша, но и стоимость проекций, доступность оптимизированной реализации и качество после обучения.

| Механизм | Кеш на токен и слой | Что разделяется или сжимается |
|---|---|---|
| MHA | $2n_h d_h$ | У каждой головы свои ключи и значения |
| MQA | $2d_h$ | Одна пара ключа и значения для всех запросов |
| GQA | $2n_g d_h$ | Одна пара ключа и значения на группу запросов |
| MLA | $d_c + d^R_h$ | Общий латентный KV-вектор и отдельный позиционный ключ |

**Таблица 1:** Число сохраняемых элементов без множителей $B$, $T$, $L$ и размера типа данных. Для MLA приведена схема DeepSeek-V2 [5].

# 5. Mixture of Experts

## 5.1. Разреженный FFN

Плотный FFN использует одни и те же параметры для каждого токена. Требуется увеличить число доступных параметров, не вычисляя все их для каждого входа. Для этого вместо одного FFN вводятся $N$ экспертов $E_1, \dots, E_N$ и маршрутизатор.

Для состояния токена $x \in \mathbb{R}^d$ маршрутизатор вычисляет логиты и вероятности

$$r(x) = W_r x \in \mathbb{R}^N, \qquad p_i(x) = \frac{\exp r_i(x)}{\sum_{j=1}^{N} \exp r_j(x)}.$$

Матрица $W_r$ обучается. Выбирается множество $S(x)$ из $k$ экспертов с наибольшими логитами. Выход разреженного слоя записывается как

$$y(x) = \sum_{i \in S(x)} g_i(x) E_i(x).$$

Веса $g_i(x)$ зависят от архитектуры. При перенормировке внутри top-$k$

$$g_i(x) = \frac{p_i(x)}{\sum_{j \in S(x)} p_j(x)}, \qquad i \in S(x).$$

В Switch Transformer используется $k = 1$, а выход выбранного эксперта масштабируется его вероятностью маршрутизатора. В других архитектурах часто выбирают несколько экспертов [7, 8]. Для остальных экспертов вычисление этого токена не выполняется.

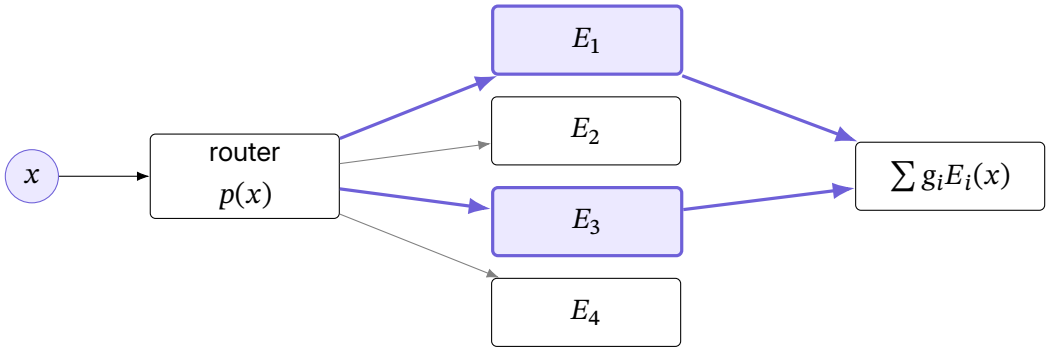



**Рис. 7:** Маршрутизация top-2 среди четырех экспертов. Для токена вычисляются только выделенные FFN.

Для одинаковых экспертов фиксированного размера общее число параметров экспертов растет примерно в $N$ раз относительно одного FFN. Число активированных параметров экспертов и операций на токен определяется $k$. Общие эксперты и эксперты разного размера учитываются отдельно. Эти величины нужно указывать явно. Фраза «модель имеет $P$ параметров» для MoE недостаточна без числа активируемых параметров.

## 5.2. Балансировка нагрузки и емкость эксперта

Если маршрутизатор отправляет большинство токенов одному эксперту, часть параметров почти не обучается, а выбранное устройство становится узким местом. Для Switch Transformer вводится вспомогательная потеря балансировки [7]. Пусть в батче $T_b$ токенов. Для эксперта $i$ определим

$$f_i = \frac{1}{T_b} \sum_{x \in \mathcal{B}} \mathbf{1}\{\arg\max_j p_j(x) = i\}, \qquad P_i = \frac{1}{T_b} \sum_{x \in \mathcal{B}} p_i(x).$$

$f_i$ - фактическая доля назначенных токенов, $P_i$ - средняя вероятность маршрутизатора. Вспомогательная потеря равна

$$\mathcal{L}_{\text{bal}} = \alpha N \sum_{i=1}^{N} f_i P_i.$$

При равномерной маршрутизации обе величины близки к $1/N$. Вектор $P$ дифференцируем по параметрам маршрутизатора, а дискретные назначения в $f$ обычно рассматриваются как фиксированные при вычислении этого градиента. Коэффициент $\alpha$ задает силу балансировки относительно основной потери модели.

На распределенном оборудовании эксперт имеет ограниченную емкость. Для top-1 маршрутизации ее часто задают как

$$C = \left\lceil \frac{T_b}{N} \gamma \right\rceil,$$

где $\gamma \geq 1$ - коэффициент запаса. При top-$k$ в числителе учитывается число назначений $kT_b$. Малый запас приводит к переполнению экспертов, большой - к пустым местам и лишним вычислениям. Конкретная реализация может отбрасывать лишние назначения, передавать токен по residual-связи или использовать маршрутизацию без отбрасывания; эти варианты нельзя смешивать при сравнении результатов.

### 5.3. Распределенное выполнение

Эксперты обычно размещаются на разных ускорителях. После маршрутизации состояния токенов пересылаются на устройства выбранных экспертов, затем результаты возвращаются. Поэтому стоимость слоя включает не только матричные умножения, но и обмен many-to-many. Несбалансированный router увеличивает время ожидания, даже если среднее число операций не меняется.

На обучении в памяти должны находиться параметры экспертов и состояния оптимизатора. На инференсе все веса тоже должны быть доступны, хотя один токен обращается только к $k$ экспертам. Разреженные вычисления не означают такого же сокращения памяти весов. Практический выигрыш зависит от размера батча, распределения токенов, топологии соединений и реализации dispatch/combine.

В DeepSeekMoE эксперты делятся на общие, применяемые ко всем токенам, и маршрутизируемые; маршрутизируемые эксперты делают более мелкими и выбирают несколько из них [9]. Это конкретный вариант MoE. Базовая схема остается той же: router задает условное вычисление, а качество и скорость зависят от балансировки и распределенного выполнения.

## 6. Положение механизмов в блоке трансформера

Рассмотренные методы изменяют разные операции и могут использоваться совместно.

1. Из состояния $h_t$ строятся запросы, ключи и значения. RoPE применяется к позиционной части запросов и ключей.
2. MHA, GQA или MLA задает, сколько представлений ключей и значений создается и что сохраняется в KV-cache.
3. После attention выполняется FFN. В MoE вместо одного плотного FFN маршрутизатор выбирает экспертов отдельно для каждого токена.

RoPE не сокращает KV-cache. GQA не увеличивает число FFN-параметров. MoE не меняет квадратичную по длине последовательности матрицу внимания. MLA уменьшает объем KV-cache, но само по себе не решает распределенную маршрутизацию экспертов. Эти методы отвечают на разные ограничения.

| Метод | Часть блока | Изменяемая величина | Основная проверка |
|---|---|---|---|
| RoPE | Оценка внимания | Позиционная зависимость $q^\top k$ | Качество на разных расстояниях и длинах |
| GQA | Attention и кеш | Число голов ключей и значений | Качество, память кеша, скорость декодирования |
| MLA | Attention и кеш | Размер латентного KV-представления | Качество, память, стоимость проекций и ядер |
| MoE | FFN | Общие и активируемые параметры | Качество, балансировка, связь и пропускная способность |

**Таблица 2:** Роль каждого механизма в блоке трансформера.

# 7. Оценка качества и эффективности

Модель с измененной архитектурой сравнивают с плотной базовой моделью при явно заданном бюджете. Для языковой модели фиксируются токенизатор, обучающие данные, число токенов обучения и процедура оценки. Если сравнивается только инференс уже обученных моделей, фиксируются длины входа и выхода, батч, числовой формат, устройство и реализация ядер.

Минимальный набор измерений:

- средняя отрицательная логарифмическая вероятность или perplexity на одном корпусе;
- метрики прикладных задач, если модель предназначена не только для language modeling;
- память весов и отдельно пиковая память KV-cache;
- время обработки входа и время одного шага декодирования;
- число сгенерированных токенов в секунду при заданном батче;
- для MoE - распределение нагрузки, доля переполненных назначений и объем межустройственного обмена.

Частые ошибки сравнения:

- сопоставлять общее число параметров MoE с активируемыми параметрами плотной модели;
- сообщать теоретическое сокращение KV-cache без фактической пиковой памяти;
- объединять prefill и decode в одно время, хотя их узкие места различаются;
- сравнивать GQA или MLA на разных ядрах attention и приписывать весь выигрыш архитектуре;
- считать увеличение максимальной длины контекста доказательством качества на длинных зависимостях.

# 8. Основные выводы

- RoPE поворачивает пары координат запросов и ключей. Их скалярное произведение зависит от относительного сдвига позиций.
- GQA оставляет много голов запросов, но уменьшает число голов ключей и значений. Размер KV-cache сокращается в $n_h/n_g$ раз относительно MHA.
- MLA в конфигурации DeepSeek-V2 хранит совместный латентный вектор ключей и значений и отдельный позиционный ключ. Его кеш определяется $d_c + d_h^R$, а не $2n_h d_h$.
- MoE увеличивает число доступных FFN-параметров, вычисляя для токена только $k$ выбранных экспертов. Для работы метода нужны балансировка и учет стоимости пересылки токенов.
- Качество и эффективность проверяются совместно. Формальное число параметров или размер кеша не заменяет измерений на одинаковых данных и оборудовании.

# A. Вывод относительного сдвига в RoPE

Для одной пары координат поворот обладает свойствами

$$R(\phi)^\top = R(-\phi), \qquad R(\phi)R(\psi) = R(\phi+\psi).$$

Пусть $q_m = R(m\theta)u_m$ и $k_n = R(n\theta)v_n$. Тогда

$$\begin{aligned} q_m^\top k_n &= u_m^\top R(m\theta)^\top R(n\theta) v_n \\ &= u_m^\top R(-m\theta) R(n\theta) v_n \\ &= u_m^\top R((n-m)\theta) v_n. \end{aligned}$$

В полной размерности равенство применяется независимо к каждой паре координат со своей частотой $\theta_r$. Позиции не добавляются к содержательным векторам, а меняют ориентацию запроса и ключа.

# B. Пример расчета KV-cache

Пусть $B = 8$, $T = 4096$, $L = 32$, $n_h = 32$, $d_h = 128$, один элемент кеша занимает $s = 2$ байта. Для MHA

$$M_{\text{MHA}} = 2 \cdot 8 \cdot 4096 \cdot 32 \cdot 32 \cdot 128 \cdot 2 = 17\,179\,869\,184 \text{ байт} \approx 16 \text{ GiB}.$$

Для GQA с $n_g = 8$:

$$M_{\text{GQA}} = M_{\text{MHA}} \frac{8}{32} \approx 4 \text{ GiB}.$$

Расчет показывает линейную зависимость от батча и длины контекста. Он не включает веса модели, активации текущего шага, служебные буферы и фрагментацию памяти, поэтому фактическое потребление будет выше.

# Литература


[1] Vaswani A. et al. Attention Is All You Need. 2017. arXiv:1706.03762.

[2] Su J. et al. RoFormer: Enhanced Transformer with Rotary Position Embedding. 2021. arXiv:2104.09864.

[3] Shazeer N. Fast Transformer Decoding: One Write-Head is All You Need. 2019. arXiv:1911.02150.

[4] Ainslie J. et al. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. 2023. arXiv:2305.13245.

[5] DeepSeek-AI. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. 2024. arXiv:2405.04434.

[6] Shazeer N. et al. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. 2017. arXiv:1701.06538.

[7] Fedus W., Zoph B., Shazeer N. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. 2021. arXiv:2101.03961.

[8] Jiang A. Q. et al. Mixtral of Experts. 2024. arXiv:2401.04088.

[9] Dai D. et al. DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models. 2024. arXiv:2401.06066.